```
#!/usr/bin/env python3
"""World Pulse collector: RSS/Atom feeds -> data.json + history.json.
Only the standard library (Python 3.9+). Run:  python collector.py
Env: OUT_DIR (default ".")"""
import html, json, os, re, sys, time
import urllib.request
from concurrent.futures import ThreadPoolExecutor
from datetime import datetime, timezone
from email.utils import parsedate_to_datetime
from xml.etree import ElementTree as ET

OUT = os.environ.get("OUT_DIR", ".")
MAX_AGE_H, PER_SOURCE, HIST_POINTS = 24, 40, 48
GN = "https://news.google.com/rss/search?q=site:{}+world+when:1d&hl=en-
US&gl=US&ceid=US:en"
SOURCES = {  # Reuters and AP have no public RSS any more, so they come via Google
News
    "BBC": "https://feeds.bbci.co.uk/news/world/rss.xml",
    "Al Jazeera": "https://www.aljazeera.com/xml/rss/all.xml",
    "DW": "https://rss.dw.com/rdf/rss-en-world",
    "France 24": "https://www.france24.com/en/rss",
    "The Guardian": "https://www.theguardian.com/world/rss",
    "NPR": "https://feeds.npr.org/1004/rss.xml",
    "Sky News": "https://feeds.skynews.com/feeds/rss/world.xml",
    "Reuters": GN.format("reuters.com"),
    "AP": GN.format("apnews.com"),
}
ORDER = list(SOURCES)

# --- tension lexicon: word (or prefix*) -> weight (negative = calming) ---
LEX = {
    "war": 3, "wars": 3, "kill*": 3, "dead": 3, "death*": 2, "massacre*": 3,
"invasion": 3, "invad*": 3,
    "bomb*": 3, "airstrike*": 3, "strike*": 2, "missile*": 3, "terror*": 3,
"attack*": 2, "shell*": 2,
    "hostage*": 3, "nuclear": 2, "troops": 2, "offensive": 2, "clash*": 2,
"explosion*": 2, "shooting*": 2,
    "crisis": 2, "sanction*": 2, "threat*": 2, "tension*": 2, "unrest": 2, "coup": 3,
"blockade": 2,
    "evacuat*": 2, "casualt*": 3, "drone*": 2, "escalat*": 3, "collapse*": 2,
"famine": 2, "earthquake*": 2,
    "tsunami": 2, "assassinat*": 3, "crackdown": 2, "ultimatum": 2, "cyberattack*":
2, "rocket*": 2,
    "gunmen": 2, "protest*": 1, "flood*": 1, "wildfire*": 1, "hurricane*": 1,
"arrest*": 1, "warn*": 1,
    "recession": 1, "inflation": 1, "refugee*": 1,
```

```
    "talks": -1, "negotiat*": -1, "agreement": -2, "agree*": -1, "ceasefire": -2,
"truce": -2, "peace*": -2,
    "deal": -1, "summit": -1, "aid": -1, "rescue*": -1, "breakthrough": -2,
"released": -1, "reopen*": -1,
    "recover*": -1, "cooperat*": -2, "treaty": -2, "welcome*": -1, "boost*": -1,
"growth": -1,
    "vaccine*": -1, "discover*": -1, "wins": -1, "celebrat*": -1, "restor*": -1,
}
EX = {k: v for k, v in LEX.items() if not k.endswith("*")}
PRE = [(k[:-1], v) for k, v in LEX.items() if k.endswith("*")]

REGIONS = {
    "Европа": "ukraine russia europe european germany france poland britain london
paris berlin moscow kyiv kremlin nato brussels italy spain serbia balkans hungary
romania czech sweden norway finland greece eu".split(),
    "Ближний Восток": "israel gaza iran iraq syria lebanon yemen saudi hamas
hezbollah houthi houthis tehran beirut qatar uae jordan turkey palestinian
palestinians".split() + ["west bank"],
    "Азия": "china japan korea india pakistan taiwan beijing tokyo myanmar thailand
vietnam philippines indonesia afghanistan asia bangladesh nepal kazakhstan".split() +
["hong kong"],
    "Африка": "africa sudan nigeria ethiopia congo somalia sahel mali niger kenya
egypt libya ghana uganda mozambique cameroon".split() + ["south africa"],
    "Америка": "us u.s. america american trump washington canada mexico brazil
argentina venezuela colombia cuba congress pentagon".split() + ["white house", "wall
street", "latin america"],
}
CATS = {
    "Конфликты": "war attack strike missile troops killed bomb ceasefire hostage
military army drone shelling terror clash invasion offensive".split(),
    "Природа": "earthquake flood wildfire hurricane storm typhoon volcano climate
heatwave drought tsunami cyclone blizzard".split(),
    "Наука": "scientists scientist study research space nasa vaccine cancer discovery
telescope robot ai technology".split(),
    "Экономика": "economy economic market markets stocks inflation trade tariff
tariffs oil bank gdp currency recession prices shares budget bitcoin dollar
jobs".split(),
    "Политика": "election president minister parliament vote government sanctions
summit talks treaty court trial protest prime".split(),
}
STOP = set("the an of to in on for and with as at by from is are was were be been
after over amid says say new news live update updates will would could may that this
its it into than more about out up down against his her their he she they who what
how why when".split())


def rx(words):
    return re.compile(r"(?<!\w)(?:%s)(?!\w)" % "|".join(re.escape(w) for w in words),
re.I)
```

```
REG_RX = {k: rx(v) for k, v in REGIONS.items()}
CAT_RX = {k: rx(v) for k, v in CATS.items()}


def tension(text):
    s = 0
    for w in re.findall(r"[a-z]+", text.lower()):
        if w in EX:
            s += EX[w]
        else:
            for p, wt in PRE:
                if w.startswith(p):
                    s += wt
                    break
    return max(0, min(100, round(38 + 9 * s)))


def best(rxs, text, default):
    sc = {k: len(r.findall(text)) for k, r in rxs.items()}
    k = max(sc, key=sc.get)
    return k if sc[k] else default


def tokens(t):
    return {w for w in re.findall(r"[a-z0-9]+", t.lower()) if len(w) > 2 and w not in
STOP}


def clean(s):
    return re.sub(r"\s+", " ", html.unescape(re.sub(r"<[^>]+>", " ", s or
"").)).strip()


def local(tag):
    return tag.rsplit("}", 1)[-1]


def parse_date(s):
    try:
        d = parsedate_to_datetime(s)
    except Exception:
        try:
            d = datetime.fromisoformat(s.strip().replace("Z", "+00:00"))
        except Exception:
            return time.time()
    if d.tzinfo is None:
        d = d.replace(tzinfo=timezone.utc)
    return d.timestamp()
```

```
def parse(raw, name):
    root, items, gn = ET.fromstring(raw), [], name in ("Reuters", "AP")
    for el in root.iter():
        if local(el.tag) not in ("item", "entry"):
            continue
        f = {}
        for c in el:
            k = local(c.tag)
            if k == "link" and c.get("href"):
                f.setdefault("link", c.get("href"))
            elif c.text and k not in f:
                f[k] = c.text
        title = clean(f.get("title"))
        if gn:
            title = re.sub(r"\s+[-—–|]\s+[^---|]{2,40}$", "", title)
        if not title:
            continue
        desc = "" if gn else clean(f.get("description") or f.get("summary") or
f.get("content"))
        ts = parse_date(f.get("pubDate") or f.get("published") or f.get("updated") or
f.get("date") or "")
        items.append({"src": name, "title": title, "desc": desc, "url":
(f.get("link") or "").strip(), "ts": ts})
    cutoff = time.time() - MAX_AGE_H * 3600
    items = [i for i in items if i["ts"] >= cutoff]
    return sorted(items, key=lambda i: -i["ts"])[:PER_SOURCE]


def fetch(name):
    err = None
    for i in range(3):
        try:
            req = urllib.request.Request(SOURCES[name], headers={"User-Agent":
"Mozilla/5.0 (compatible; WorldPulse/1.0)"})
            with urllib.request.urlopen(req, timeout=20) as r:
                return name, parse(r.read(), name)
        except Exception as e:
            err = e
            time.sleep(1.5 * (i + 1))
    print(f"[warn] {name}: {err}", file=sys.stderr)
    return name, []


def same(a, b):
    c = len(a & b)
    if not c:
        return False
    j = c / len(a | b)
    return (c >= 3 and j >= 0.25) or (c >= 2 and j >= 0.5)
```

```
def cut(s, n):
    if len(s) <= n:
        return s
    return s[:n].rsplit(" ", 1)[0].rstrip(",.;:") + "…"


def build(by_src, history=None, now=None):
    now = now or time.time()
    allitems = [i for v in by_src.values() for i in v]
    if len(allitems) < 10:
        raise RuntimeError("too few headlines collected")
    for i in allitems:
        i["text"] = i["title"] + " " + i["desc"]
        i["ten"] = tension(i["title"])
        i["tok"] = tokens(i["title"])
        i["reg"] = best(REG_RX, i["text"], "Мир")
    # clustering
    clusters = []
    for it in sorted(allitems, key=lambda i: -i["ts"]):
        for c in clusters:
            if same(it["tok"], c["tok"]):
                c["m"].append(it)
                break
        else:
            clusters.append({"tok": it["tok"], "m": [it]})
    events = []
    for c in clusters:
        m = c["m"]
        srcs = sorted({i["src"] for i in m}, key=ORDER.index)
        age_h = max(0.0, (now - max(i["ts"] for i in m)) / 3600)
        ten = round(sum(i["ten"] for i in m) / len(m))
        imp = min(100, round(25 * len(srcs) + 25 * max(0, 1 - age_h / MAX_AGE_H) +
ten / 10))
        rep = min(m, key=lambda i: (ORDER.index(i["src"]), -len(i["desc"])))
        joined = " ".join(i["text"] for i in m)
        other = next((i["title"] for i in m if i is not rep and i["title"] !=
rep["title"]), "")
        events.append({
            "t": cut(rep["title"], 140), "x": cut(rep["desc"] or other, 240),
            "r": best(REG_RX, joined, "Мир"), "c": best(CAT_RX, joined, "Общее"),
            "imp": imp, "ten": ten, "ago": round(age_h * 60), "s": srcs,
            "u": rep["url"] if rep["url"].startswith("http") else "",
        })
    multi = sorted((e for e in events if len(e["s"]) > 1), key=lambda e: -e["imp"])
[:10]
    if len(multi) < 6:
        rest = sorted((e for e in events if len(e["s"]) == 1), key=lambda e: -
e["imp"])
```

```
        multi += rest[: 6 - len(multi)]
    for n, e in enumerate(multi, 1):
        e["id"] = n
    # sources, regions, index
    srcv = []
    for n in ORDER:
        its = by_src.get(n) or []
        if its:
            srcv.append({"n": n, "v": round(sum(i["ten"] for i in its) / len(its))})
    media = round(sum(s["v"] for s in srcv) / len(srcv))
    regv = []
    for r in REGIONS:
        its = [i for i in allitems if i["reg"] == r]
        if len(its) >= 3:
            regv.append({"n": r, "v": round(sum(i["ten"] for i in its) / len(its))})
    hist = [h for h in (history or []) if now * 1000 - h["t"] < 26 * 3600 * 1000]
    hist = (hist + [{"t": round(now * 1000), "v": media}])[-HIST_POINTS:]
    last = [h["v"] for h in hist[-6:]]
    smooth = round(sum(last) / len(last))
    return {
        "updated": round(now * 1000), "demo": False, "claude_delta": 0,
        "second_delta": smooth - media, "second_label": "Сглаженный индекс",
        "second_note": "среднее за последние 3 часа",
        "sources": srcv, "regions": regv, "events": multi, "history": hist,
    }


def write(name, obj):
    p = os.path.join(OUT, name)
    with open(p + ".tmp", "w", encoding="utf-8") as f:
        json.dump(obj, f, ensure_ascii=False, indent=1)
    os.replace(p + ".tmp", p)


def main():
    with ThreadPoolExecutor(max_workers=len(SOURCES)) as ex:
        by_src = dict(ex.map(fetch, SOURCES))
    ok = [n for n, v in by_src.items() if v]
    print(f"[info] sources ok: {len(ok)}/{len(SOURCES)} ({', '.join(ok)})")
    try:
        with open(os.path.join(OUT, "history.json"), encoding="utf-8") as f:
            history = json.load(f)
    except Exception:
        history = []
    try:
        data = build(by_src, history)
    except RuntimeError as e:
        print(f"[error] {e}; data.json left untouched", file=sys.stderr)
        sys.exit(1)
    write("history.json", data["history"])
```

```
    write("data.json", data)
    print(f"[info] media index {data['sources'] and round(sum(s['v'] for s in
data['sources'])/len(data['sources']))}%, events {len(data['events'])}")


if __name__ == "__main__":
    main()
```